



Manipulación y Gestión de Datos

Clase 2 - Creación y manipulación de tablas (Parte II)



¿Qué aprenderás en esta sesión?

Aprenderemos a insertar datos manualmente en tablas previamente creadas utilizando la instrucción `INSERT INTO`, comprendiendo sus variantes, sintaxis y buenas prácticas.

Exploraremos escenarios productivos reales donde el ingreso correcto de datos garantiza la consistencia de la información, el cumplimiento de restricciones y la trazabilidad en los sistemas.

Evaluaremos también cómo validar que los datos insertados cumplen con las reglas establecidas en el diseño de la base de datos.

Desafío inicial

- ① Formas parte del equipo de datos de una empresa que acaba de finalizar el diseño estructural de su base de datos para gestionar procesos de formación interna. Ahora es necesario poblar las tablas con registros de personas participantes, cursos y sus respectivas inscripciones. Para asegurar la operatividad del sistema, deberás insertar registros de manera manual cumpliendo con todas las restricciones previamente definidas, como claves primarias, tipos de datos correctos y valores no nulos.

¿Qué consideraciones debes tener al ingresar manualmente datos en una tabla relacional?

Para garantizar que se cumplan todas las reglas de integridad y evitar errores durante la operación del sistema:

Para resolverlo debes poder ejecutar una serie de instrucciones INSERT INTO sobre las tablas previamente creadas, verificando que cada valor ingresado respete las restricciones estructurales definidas. Evaluar los errores que puedan surgir, justificar su causa, y realizar las correcciones necesarias para lograr una inserción exitosa y conforme a los requerimientos técnicos.



Inserción manual de valores en una tabla

¿Qué errores podrían surgir al insertar datos manualmente en una tabla mal diseñada o con restricciones no consideradas?

Introducción al INSERT INTO

La instrucción INSERT INTO permite agregar nuevos registros a una tabla existente en una base de datos relacional. Esta acción es fundamental para poblar tablas luego de su creación y para mantener la información actualizada.

```
INSERT INTO nombre_tabla (columna1, columna2, ...)
VALUES (valor1, valor2, ...);
```

Este comando puede utilizarse en su forma explícita (indicando las columnas) o implícita (si se insertan todos los valores en orden).

Variantes de INSERT INTO

Existen distintas formas de usar INSERT INTO según la necesidad:



Forma completa (especificando columnas)

Es la forma más segura y clara.



Forma abreviada (sin columnas)

Solo recomendable si se insertan todos los valores en el mismo orden en que fueron definidas las columnas.



Múltiples inserciones en una sola instrucción

Mejora el rendimiento cuando se insertan muchos registros.

```
INSERT INTO cursos (nombre_curso, duracion)
VALUES
  ('SQL Básico', 20),
  ('Python para Análisis', 30),
  ('Estadística Aplicada', 25);
```

Inserciones con valores por defecto y nulos

Es posible omitir columnas en la inserción si:

- Tienen un valor por defecto (DEFAULT).
- Permiten valores nulos (NULL).

```
INSERT INTO participantes (nombre, correo)
VALUES ('Elena Fuentes', 'elena.fuentes@email.com');
```

Inserciones condicionales o seguras

Antes de insertar, puede validarse que un dato no exista ya en la tabla:

Verificar existencia previa

```
SELECT COUNT(*)  
FROM participantes  
WHERE correo = 'camila.soto@email.com';
```

En sistemas robustos, esta validación se gestiona desde la capa de aplicación o mediante control de errores (try-catch) en procedimientos almacenados.

Contexto productivo

En entornos productivos, la inserción manual:

Interfaces administrativas

Se realiza desde interfaces administrativas o scripts de carga controlada.

Validaciones

Debe contemplar validaciones previas y posterior verificación.

Procesos ETL

Puede formar parte de procesos ETL (Extract, Transform, Load) para bases maestras.

Ejemplo aplicado ampliado

Consideremos que tenemos la tabla cursos:

```
CREATE TABLE cursos (  
    id_curso SERIAL PRIMARY KEY,  
    nombre_curso VARCHAR(100) NOT NULL,  
    duracion INT DEFAULT 20 CHECK (duracion > 0)  
);
```

```
INSERT INTO cursos (nombre_curso, duracion)  
VALUES ('SQL Intermedio', 25);
```

```
-- Inserción con DEFAULT  
INSERT INTO cursos (nombre_curso)  
VALUES ('Excel para negocios');
```

Buenas prácticas

Recomendaciones

- Documentar cada inserción realizada (qué, cuándo, quién).
- Asegurar respaldo antes de inserciones masivas.
- Definir DEFAULT y CHECK adecuados para evitar errores comunes.
- Incluir validaciones lógicas previas al INSERT cuando sea necesario.

Errores comunes ampliados

- Omitir columnas requeridas (NOT NULL).
- Ingresar datos duplicados en campos UNIQUE.
- Inserciones fuera de rango o con tipos incorrectos.
- Orden incorrecto de valores respecto a columnas.

Diagnóstico de errores típicos en inserciones

Error	Causa probable	Solución sugerida
Violación de NOT NULL	Falta de valor obligatorio	Asegurar datos completos
Duplicado en UNIQUE	Repetición de un identificador o correo	Verificar unicidad antes de insertar
CHECK fallido	Edad menor a permitida, duración negativa	Corregir dato o lógica de negocio

Tabla 1. Diagnóstico de errores típicos en inserciones

Fuente: Desafío Latam

Como hemos visto, insertar datos en una tabla no es simplemente cargar valores, sino respetar una estructura definida, validada y coherente. Esto permite garantizar consistencia, trazabilidad y cumplimiento de reglas de negocio.

Actividad guiada:

Inserción controlada de registros en una tabla de cursos



Objetivo de la actividad

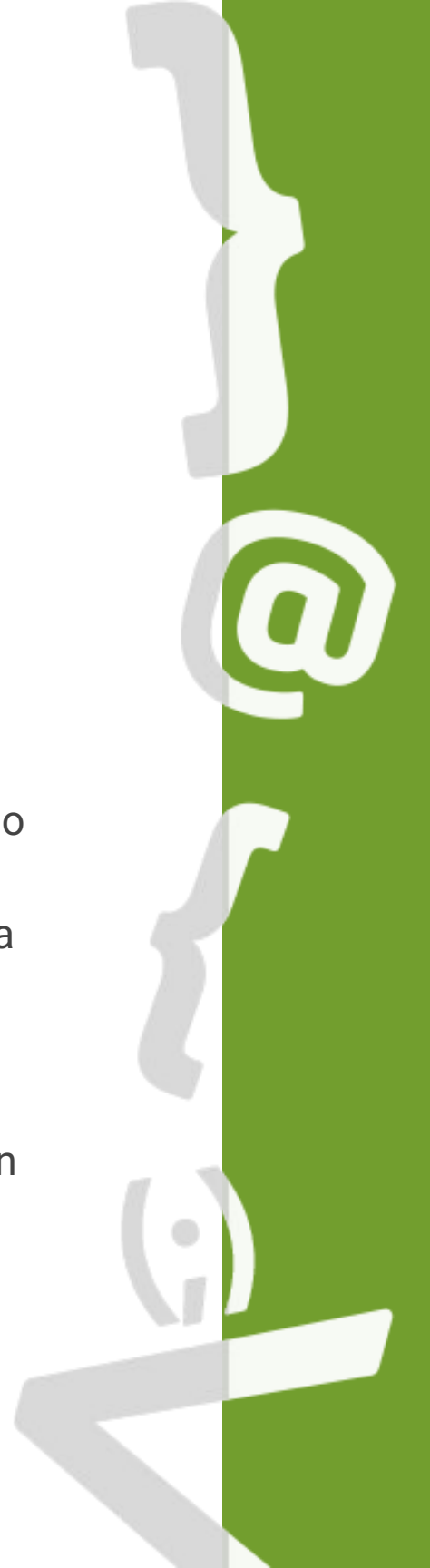
Aplicar inserciones manuales en una tabla relacional con distintas restricciones, evaluando el comportamiento del sistema ante registros válidos e inválidos, y reforzando la comprensión del uso de INSERT INTO.

Situación problema

El equipo de gestión académica de una organización educativa ha solicitado registrar los cursos que ofrecerán en el próximo semestre.

Para ello, deben ingresar la información en una base de datos que ya ha sido estructurada con restricciones que aseguran la calidad de los datos. Sin embargo, algunos intentos de inserción han generado errores que necesitan ser entendidos y corregidos.

Quienes participan deben realizar inserciones válidas e inválidas, analizar los resultados y proponer soluciones que respeten las restricciones establecidas.



Instrucciones

Ejecuta la siguiente instrucción de creación de tabla:

```
CREATE TABLE cursos (  
  id_curso SERIAL PRIMARY KEY,  
  nombre_curso VARCHAR(100) NOT NULL,  
  duracion INT DEFAULT 20 CHECK (duracion > 0),  
  modalidad VARCHAR(20) DEFAULT 'Online' CHECK (modalidad IN ('Online', 'Presencial'))  
);
```

Realiza las siguientes inserciones:

```
-- Inserción correcta  
INSERT INTO cursos (nombre_curso, duracion, modalidad)  
VALUES ('Introducción a SQL', 30, 'Presencial');  
  
-- Inserción con error: duración negativa  
INSERT INTO cursos (nombre_curso, duracion)  
VALUES ('Curso Express', -5);  
  
-- Inserción con error: modalidad inválida  
INSERT INTO cursos (nombre_curso, modalidad)  
VALUES ('Curso Avanzado', 'Híbrido');  
  
-- Inserción con valores por defecto  
INSERT INTO cursos (nombre_curso)  
VALUES ('Curso Libre');
```



Análisis de errores

Analiza los mensajes de error que se generen. Para cada uno:

- Explica por qué ocurrió.
- Describe cómo solucionarlo respetando las restricciones.

Materiales necesarios

- Editor SQL (local o en línea).
- Cuaderno de anotaciones o planilla compartida para registrar observaciones.

Insumos disponibles

- Código de creación de tabla.
- Código SQL para inserciones válidas e inválidas.



Actividad práctica autónoma:

**Registro manual de cursos
en base de datos relacional**



Objetivo de la actividad

Aplicar inserciones manuales controladas en una tabla relacional con múltiples restricciones, identificando y corrigiendo errores de ingreso mediante la validación de las condiciones estructurales de la tabla.

Contexto

Formas parte del equipo de soporte de datos de una institución que ofrece cursos de formación continua.

Te han encargado poblar manualmente la base de datos con información de nuevos cursos.

Para ello, deberás registrar los datos en una tabla con restricciones de integridad como claves primarias, campos obligatorios, valores permitidos y validaciones de rango.

Es fundamental que cada inserción cumpla con las reglas definidas por el equipo de arquitectura de datos.



Instrucciones para desarrollar la actividad

Crea la siguiente tabla en tu entorno de SQL:

```
CREATE TABLE cursos (  
  id_curso SERIAL PRIMARY KEY,  
  nombre_curso VARCHAR(100) NOT NULL,  
  duracion INT DEFAULT 20 CHECK (duracion > 0),  
  modalidad VARCHAR(20) DEFAULT 'Online' CHECK (modalidad IN ('Online', 'Presencial'))  
);
```

Inserta manualmente los siguientes cursos en la tabla, uno por uno, y observa el resultado:

```
-- Curso 1: válido  
INSERT INTO cursos (nombre_curso, duracion, modalidad)  
VALUES ('Power BI para análisis', 24, 'Online');  
  
-- Curso 2: duración incorrecta  
INSERT INTO cursos (nombre_curso, duracion)  
VALUES ('Visualización avanzada', -10);  
  
-- Curso 3: modalidad no permitida  
INSERT INTO cursos (nombre_curso, modalidad)  
VALUES ('Metodologías ágiles', 'Virtual');  
  
-- Curso 4: inserción mínima con valores por defecto  
INSERT INTO cursos (nombre_curso)  
VALUES ('SQL aplicado a negocios');
```



Análisis y corrección de errores

Por cada instrucción que genere error:

- Explica qué regla fue vulnerada.
- Propón una corrección válida para que la inserción sea exitosa.
- Ejecuta la corrección y verifica el registro.

Completa la siguiente tabla con tus observaciones (puedes crearla en Excel o en papel):

Curso	¿Se insertó correctamente?	Restricción afectada	Solución aplicada
Power BI para análisis			
Visualización avanzada			
Metodologías ágiles			
SQL aplicado a negocios			

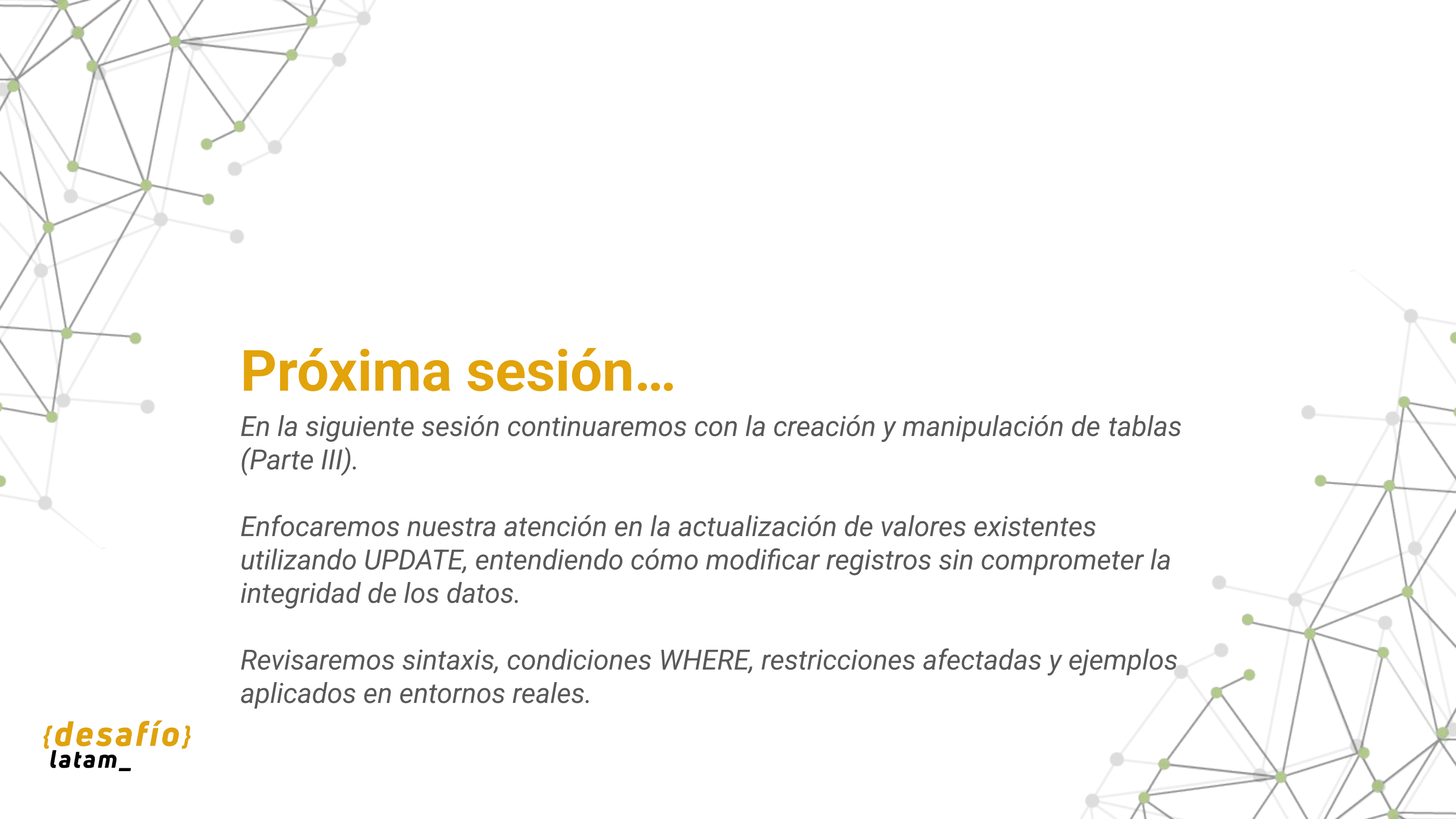
Resumen

Comprendimos el uso de INSERT INTO para agregar registros a una tabla relacional.

Exploramos las distintas variantes del comando, incluyendo inserciones múltiples y uso de valores por defecto.

Aplicamos buenas prácticas para validar datos antes de insertarlos.

Identificamos y solucionamos errores comunes asociados a las restricciones de integridad.



Próxima sesión...

En la siguiente sesión continuaremos con la creación y manipulación de tablas (Parte III).

Enfocaremos nuestra atención en la actualización de valores existentes utilizando UPDATE, entendiendo cómo modificar registros sin comprometer la integridad de los datos.

Revisaremos sintaxis, condiciones WHERE, restricciones afectadas y ejemplos aplicados en entornos reales.

Preguntas de cierre

- 1 ¿Cuál es la diferencia entre usar INSERT INTO con y sin especificar columnas?
- 2 ¿Qué problemas puede generar una inserción sin considerar los valores por defecto o restricciones CHECK?
- 3 ¿Qué papel cumplen las validaciones previas a la inserción en entornos productivos?
- 4 ¿Cómo podrías prevenir la duplicación de registros al insertar manualmente?
- 5 ¿Por qué es importante documentar las inserciones realizadas en una base de datos?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2022). Database Systems: Design, Implementation, & Management (14.^a ed.). Cengage Learning.
- Oppel, A. J. (2021). SQL: A Beginner's Guide (5.^a ed.). McGraw-Hill Education.
- Rob, P., & Coronel, C. (2019). Database Principles: Fundamentals of Design, Implementation, and Management (10.^a ed.). Cengage Learning.
- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. <https://www.postgresql.org/docs/>